



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 5 PolyRISC

Florian DELHON

Hiver 2026

Introduction

À la fin de ce laboratoire, vous devrez être capable de :

- Analyser le chemin des données d'un processeur à usage général afin d'estimer les ressources requises, déterminer les limites opérationnelles du processeur, proposer des changements architecturaux pour implémenter de nouvelles opérations, et donner le code VHDL modifié
- Proposer des modifications à l'unité de contrôle et à l'encodage des instructions pour ajouter de nouvelles instructions à un processeur à usage général, et donner le code VHDL modifié

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Concepts couverts dans les **Laboratoires 0, 1, 2, 3 et 4**
2. Matière du cours « **Thème 07 - Vérification de systèmes numériques** »
3. Matière du cours « **Thème 09 - Processeur à usage général** »

Liste des fichiers

Fichier	Contenu
labo5.ipynb	<u>Notebook faisant office de cahier de réponses interactif à rendre</u>
src/PolyRISC_v2.vhd src/PolyRISC_ROM.vhd src/PolyRISC_utils_pkg.vhd	Implémentation du PolyRISC en VHDL, avec sa mémoire d'instruction et ses utilitaires
src/PolyRISC_tb.vhd	Banc d'essai pour tester le bon fonctionnement du PolyRISC et du programme
src/AXI_stream_ROM.vhd	Convertisseur AXI-Stream pour écrire la mémoire d'instruction depuis un notebook Jupyter
src/power_on_reset.vhd	Module de réinitialisation au démarrage
src/top.vhd	<i>Top-level</i> du projet
src/fibonacci_n.txt	Fichier de vecteurs de test pour le banc d'essai
src/fibonacci_res.txt	Fichier des réponses attendues pour chaque entrée du src/fibonacci_n.txt
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts <i>wrappant</i> le <i>top level</i> dans un design complet pour la plateforme PYNQ
racine_recherche_dicho.ods	Classeur simulant la recherche dichotomique

Séance de laboratoire

Une séance de 3 heures sera dédiée à la réalisation de ce laboratoire : le **2 avril 2026 pour le groupe 2**, et le **16 avril 2026 pour le groupe 1**. Elle aura lieu à **8h30**, dans la salle **L-3714**.

Remise de code

Une remise de votre code est demandée sur votre dépôt **GitHub**. Pour ce faire, un *push* final contenant l'ensemble des fichiers nécessaires doit être fait avant le **19 avril 2026 à 22h, pour les deux groupes**. Vous êtes libre de faire les *push* intermédiaires souhaités avant cette date. Il vous est demandé de **ne pas modifier les noms des fichiers/entités/architectures, ni la liste des generic/port**.

Entrevue/démonstration

Pour ce dernier laboratoire, **aucune entrevue** ne sera réalisée. Assurez-vous donc de remettre tous les fichiers demandés, et de compléter intégralement le cahier de réponses labo5.ipynb.

Barème

Ce laboratoire sera noté sur 10, et comptera pour 2% de votre note finale.

Critères	Points
Remise de code	10
Partie 1	2
Partie 2	2
Partie 3	4
Partie 4	2
NB : Pour chaque partie, la notation s'appuiera sur le code rendu ainsi que sur les réponses écrites dans le labo5.ipynb	
Entrevue/démonstration	-
Aucune entrevue ne sera réalisée	-
Total	10

! Évaluation

Pour vous aider à identifier les points évalués, vous pourrez vous fier aux encadrés rouges comme celui-ci. Veillez à bien les lire afin d'obtenir la note maximale.

Partie 0 : Préparation de l'environnement de travail

Initialisation du projet Vivado

Grâce aux sources fournies dans votre dépôt de départ, **initialisez un nouveau projet Vivado**. La procédure est détaillée dans la **Note 1**.

Initialisation du *notebook* Jupyter

Dès **maintenant**, allumez votre PYNQ-Z2, connectez-vous au serveur Jupyter, créez un dossier, mettez-y votre *notebook* `labo5.ipynb` (et le dossier `images`) et ouvrez-le.

Upload et Download

Durant la réalisation du laboratoire, vous aurez à faire plusieurs allers-retours entre la PYNQ-Z2 et votre ordinateur. Sur l'interface Jupyter, pensez donc bien à utiliser :

- le bouton **Upload** pour téléverser vos *bitstreams*, *hardware-handoffs*, ou images, afin d'avoir un **set-up matériel fonctionnel** ;
- le bouton **Download** pour télécharger tous vos fichiers sur votre ordinateur, afin de tenir votre **dépôt Git à jour**.

Partie 1 : Découverte du PolyRISC

Contexte

Ce travail est la suite et fin des **Laboratoires 3** et **4**, où vous aviez implémenté deux circuits dédiés au calcul de la racine carrée. Dans ce dernier laboratoire, vous allez mettre en place ce calcul sur un processeur RISC-V modélisé en VHDL et développé à Polytechnique : le PolyRISC. Vu le temps de pratique limité, l'architecture complète du PolyRISC vous est fournie. Après l'avoir comprise, vous procéderez en 4 temps. Dans la **Partie 1**, vous prendrez en main le PolyRISC ainsi que son flot de validation pour un programme de démonstration calculant la suite de Fibonacci. Dans la **Partie 3**, vous mettrez en place le calcul de la racine carrée par recherche dichotomique, sachant que cela impliquera au préalable l'ajout de nouvelles opérations sur le PolyRISC dans la **Partie 2**. Et dans la **Partie 4**, vous ferez la validation matérielle du calcul de la racine carrée sur ce PolyRISC étendu, en l'implémentant sur FPGA et en le programmant en langage machine.

Architecture du PolyRISC

Le PolyRISC est un processeur à usage générale RISC (*Reduced Instruction Set Computer*). Son architecture interne est présentée à la Figure 1 à la page suivante. Tous les détails de ce processeur sont présentés dans les diapositives du « **Thème 09 - Processeur à usage général** ».

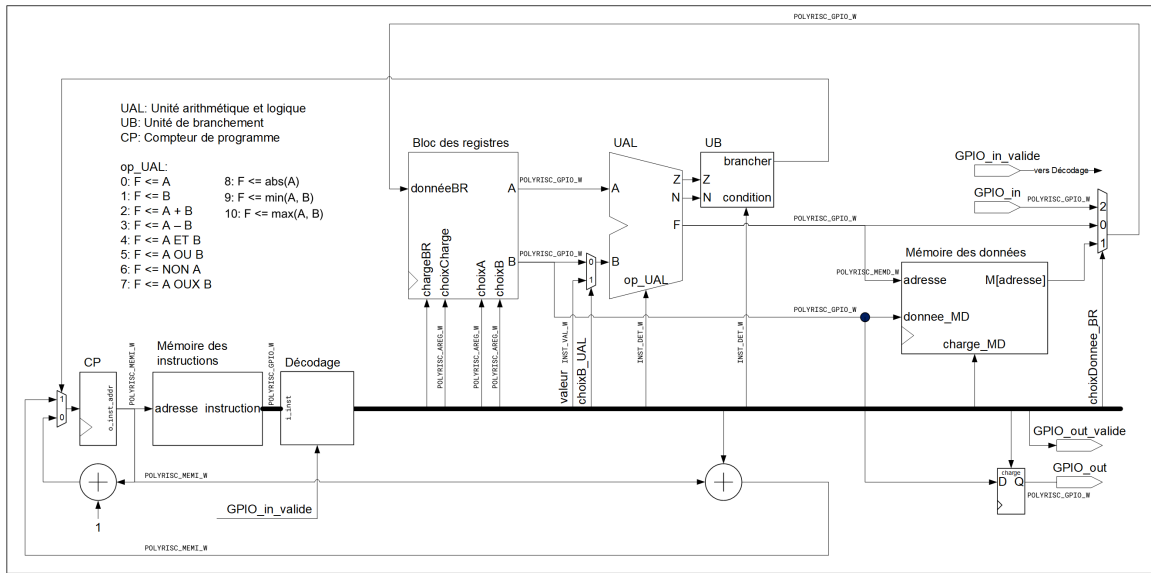


Fig. 1. – Schéma de l'architecture du PolyRISC

Même si le schéma montre une mémoire des instructions interne, elle est ici **externe** pour les besoins du laboratoire. Le tout est décrit dans les fichiers suivants :

- `src/PolyRISC_v2.vhd` constitue la description principale des **chemins de données et de contrôle** du PolyRISC,
- `src/PolyRISC_utils_pkg.vhd` est le *package* VHDL définissant les **constantes**, les **types**, et certaines fonctions **utilitaires**,
- `src/PolyRISC_ROM.vhd` est la description de la **mémoire d'instructions**.

En terme de communications, le *notebook* `labo5.ipynb` entre en détails sur les protocoles en place dans sa **Partie 0**. L'essentiel à retenir est que deux protocoles sont en jeu :

- **AXI-Stream** est utilisé pour **flasher la mémoire d'instructions** du PolyRISC afin de le programmer,
- **AXI-GPIO** est utilisé pour **interagir avec le PolyRISC** (appliquer une entrée, lire la sortie) et **écrire les signaux de contrôle**.

Enfin, notez que nous considérerons un chemin de données de **32 bits** de large, un bloc de **16 registres**, une mémoire de données de **256 mots**, une mémoire des instructions de **256 mots de 32 bits**.

Exemple : Suite de Fibonacci

Pour prendre en main le flot de validation du PolyRISC, vous vous exercerez d'abord sur un programme calculant le **n -ième nombre de la suite de Fibonacci** (F_n). Le pseudo-assembleur¹ est présenté à la page suivante (voir Pseudo-code 1), ainsi que l'utilisation du bloc de registres (voir Tableau 1).

¹Le signe « dollar » (\$) indique une constante.

```

R0 := GPIO_in
R1 := $0
R3 := $0
R4 := $1
si R1 = R0 aller à CP + 6
R5 := R3 + R4
R3 := R4
R4 := R5
R1 := R1 + $1
si R1 <= R0 aller à CP + -4
GPIO_out := R3

```

Pseudo-code 1. – Calcul des nombres de Fibonacci en pseudo-assembleur

Registre	Contenu
fichierRegistres[0]	Entrée n
fichierRegistres[1]	Rang courant k dans la suite de Fibonacci
fichierRegistres[2]	-
fichierRegistres[3]	Nombre F_n
fichierRegistres[4]	Nombre F_{n+1}
fichierRegistres[5]	Accumulateur

Tableau 1. – Utilisation des registres dans le pseudo-assembleur de démonstration

Simulation

Le banc d'essai `PolyRISC_tb.vhd` fournit ce qu'il vous faut pour tester ce programme en simulation. Observez sa structure, en particulier la manière dont il programme le PolyRISC et lit ses vecteurs de test dans un fichier `fibonacci_n.txt` (donné dans le code de départ). **Renseignez le chemin exact** du `fibonacci_n.txt` sur votre ordinateur dans le banc d'essai (ligne 108). Ensuite, **exécutez le banc d'essai** `PolyRISC_tb.vhd`, sachant qu'un « Run All » pourra être nécessaire.

! Évaluation

- Dans le `labo5.ipynb`, répondez à la **Question 1**.
- Dans le dépôt `Git`, *push*ez votre **capture d'écran** dans le dossier `images`.

Écriture des résultats dans un fichier

Nous allons chercher à avoir une sauvegarde plus efficace des résultats. **Modifiez** votre banc d'essai `PolyRISC_tb.vhd` pour qu'il écrive **les résultats dans un fichier** `fibonacci_out.txt` (voir **Thème 07 - Vérification de systèmes numériques**). Ensuite, exécutez-le de nouveau, en prenant soin d'**aller jusqu'au bout de la simulation**.

! Évaluation

- Dans le **labo5.ipynb**, répondez à la **Question 2**.
- Dans le dépôt **Git**, *pushez* votre `fibonacci_out.txt` dans le dossier `src`.

Validation sur la carte

À la manière du **Laboratoire 4**, générez le *bitstream* puis téléversez sur la PYNQ :

- le `top.bit`, généré dans `<project_dir>/<project_name>.runs/impl_1/`,
- le `zynq.hwh`, généré dans `<project_dir>/<project_name>.gen/sources_1/bd/zynq/hw_handoff/`, et que vous renommerez **top.hwh**.

! Évaluation

- Dans le **labo5.ipynb**, répondez à la **Question 3**.

Partie 2 : Extension du PolyRISC

Ajout de la multiplication ($RC := RA * RB$)

Comme éludé en introduction, le calcul de la racine carrée reposera ici sur une **recherche dichotomique**. Au préalable, il faut **étendre l'UAL** (Unité Arithmétique et Logique) pour qu'il supporte deux opérations supplémentaires, à commencer par la **multiplication**. Vu que le produit de deux nombres sur N bits peut atteindre $2N$ bits, et que la largeur des données du PolyRISC est définie par `POLYRISC_GPIO_W`, vous réaliserez une **multiplication des `POLYRISC_GPIO_W / 2` bits de poids faible** de `RA` et `RB`. Pour accomplir cet ajout de manière propre, vous aurez à :

- définir une constante `AmulB` de valeur 11 dans le `PolyRISC_utils_pkg.vhd`,
- ajouter cette multiplication à l'UAL dans le `PolyRISC_v2.vhd`.

Ajout de la division par 2 ($RC := RA / 2$)

La seconde opération nécessaire sera la **division par 2**. Donc, de la même manière :

- définissez une constante `Adiv2` de valeur 12 dans le `PolyRISC_utils_pkg.vhd`,
- ajoutez cette division par 2 à l'UAL dans le `PolyRISC_v2.vhd`.

! Évaluation

- Dans le **labo5.ipynb**, répondez à la **Question 4**.
- Dans le dépôt **Git**, *pushez* vos `PolyRISC_v2.vhd` et `PolyRISC_utils_pkg.vhd` fonctionnels, vous n'aurez plus à y toucher.

Simulation puis validation sur la carte

Compte tenu de la faible ampleur des ajouts, il n'est pas demandé de modifier le banc d'essai. **Relancez simplement la simulation de Fibonacci** pour vous assurer que vous n'avez **rien cassé**. Ensuite, comme dans la partie précédente, générez le *bitstream* puis téléversez sur la PYNQ les nouveaux `top.bit` et `top.hwh`.

! Évaluation

- Dans le `labo5.ipynb`, répondez à la **Question 5**.
- Dans le dépôt **Git**, *pushez* vos `top.bit` et `top.hwh` à la racine, vous n'aurez plus à y toucher.

Partie 3 : Simulation de la recherche dichotomique

Algorithme

L'algorithme de recherche dichotomique est présenté au Pseudo-code 2. Notez que tel qu'implémenté, il peut seulement déterminer une approximation pour les racines s'exprimant sur 16 bits en signé, soit des valeurs inférieures à $0x7FFF^2 = 32767^2$. Le classeur `racine_recherche_dicho.ods` ([source](#)) vous permet de jouer avec différentes entrées, en observant le déroulement du calcul.

```
rechercheDichotomique(nombre):  
    haut = 0x7FFF  
    bas = 0  
    compteur = 16  
    tant que compteur > 0:  
        pivot = (haut + bas) / 2  
        carre = pivot * pivot  
        si carre > nombre:  
            haut = pivot  
        sinon:  
            bas = pivot  
        compteur = compteur - 1  
    retourner pivot
```

Pseudo-code 2. – Algorithme de recherche dichotomique (crédits : Pr. [Jean-Pierre David](#))

Pseudo-assembleur

Avant de tester ce programme sur le PolyRISC, la première étape consiste à **décrire le pseudo-assembleur** ainsi que l'**utilisation des registres** (à la manière du Pseudo-code 1 et du Tableau 1 pour Fibonacci). Évidemment, vous êtes désormais libre d'utiliser nos nouvelles opérations `RC := RA * RB` et `RC := RA / 2`.

! Évaluation

- Dans le `labo5.ipynb`, répondez à la **Question 6**.

Simulation

Connaissant ce pseudo-assembleur, **modifiez le banc d'essai** `PolyRISC_tb.vhd` pour qu'il simule le programme de recherche dichotomique. Pour ce faire :

- modifiez `memoire_inst` pour qu'il contienne **votre programme**,
- modifiez vos **fichiers d'entrée et de sortie** pour qu'ils soient `dichotomie_vecs.txt` et `dichotomie_out.txt` respectivement,
- dans un fichier `dichotomie_vecs.txt`, créez vos **vecteurs de test** en écrivant **au moins 20 valeurs** entre 0 et $0x7FFF^2$ (une par ligne).

Exécutez enfin la simulation, toujours en prenant soin d'**aller jusqu'au bout**.

! Évaluation

- Dans le `labo5.ipynb`, répondez à la **Question 7**.
- Dans le dépôt **Git**, *pushez* :
 - votre `PolyRISC_tb.vhd`, vous n'aurez plus à y toucher,
 - votre **capture d'écran** dans le dossier `images`,
 - vos `dichotomie_vecs.txt` et `dichotomie_out.txt` dans le dossier `src`.

Partie 4 : Implémentation de la recherche dichotomique

Validation sur la carte

Enfin, vous pouvez effectuer la validation que la recherche dichotomique fonctionne sur votre PolyRISC « matériel » étendu ! Comme pour Fibonacci, cela implique :

- l'écriture du **programme en langage machine**,
- la définition des **vecteurs de tests** (les mêmes que dans votre banc d'essai),
- l'exécution du **test matériel** grâce aux fonctions utilitaires fournies.

! Évaluation

- Dans le `labo5.ipynb`, répondez aux **Questions 8, 9 et 10**.
- Dans le dépôt **Git**, *pushez* votre `labo5.ipynb` **COMPLET**.

Félicitations ! Voilà qui termine les laboratoires d'INF3500 et votre découverte du monde du développement FPGA.